Supervised Learning

Foundations and Linear Models

Ye Kyaw Thu
Lab Leader, Language Understanding Lab., Myanmar

March 21, 2026

Lecture Roadmap

1. Introduction and Notation
2. 1D Linear Regression
3. The Loss Function
4. Python Implementation: Linear Regression
5. Testing and Generalization
6. Modern Trends in Supervised Learning

Introduction

- Supervised Learning, Unsupervised Learning, Semi-supervised Learning, Reinforcement Learning etc.
- ဒီနေ့တော့ Supervised Learning ဆိုတာနဲ့ ပတ်သက်ပြီး လေ့လာကြရအောင်

What is Supervised Learning?

- A mapping from one or more inputs to one or more outputs. (ဆိုလိုတာက one–to–one, one–to–many, many–to–one, many–to–many မျိုးစုံ ဖြစ်နိုင်တယ်)
- Learning from labeled examples (Ground Truth). ဒါကြောင့် လေဘယ်ထိုးထားတဲ့ ဒေတာ ရှိရမယ် သို့မဟုတ် အဖြေမှန်တွေကို ကြိုတင် ပြင်ဆင်ထားဖို့ လိုအပ်တယ်။
- **Inference:** Computing outputs from new, unseen inputs.
- **Example:** Predicting car prices based on age and mileage.
- မြန်မာစာ NLP နဲ့ ဆိုင်တဲ့ ဥပမာကို ပြရရင်...
  - **Input:** ကျောင်းမှန်မှန်တက်ပါ **Output:** ကျောင်း မှန်မှန် တက် ပါ
  - **Input:** ကျွန်တော်၏စာအုပ် **Output:** ကျွန်တော်/n ၏/ppm စာအုပ်/n

What is Supervised Learning?

- တခြား မော်ဒယ်တွေလိုပါပဲ data preparation, training, testing နဲ့ evaluation လုပ်ကြရပါတယ်
- ဒေတာပြင်တဲ့ အခါမှာ raw data ကနေ တိုက်ရိုက် train လုပ်တာမျိုး မဟုတ်လို့ လေဘယ်ထိုးကြရပါတယ် တနည်းအားဖြင့် reference ရှိရပါတယ်။
- Training လုပ်တယ် ဆိုတာကို မော်ဒယ် ဆောက်တယ်လို့လည်း ခေါ်ကြတယ်။ သင်္ချာစကားနဲ့ ပြောရင် ဖော်မြူလာ တစ်ခုခုကို သုံးပြီးတော့ mapping လုပ်တယ်၊ အဖြေမှန်ကို prediction လုပ်တယ်ပေါ့
- မော်ဒယ်ဆောက်တယ် ဆိုတာ training ဒေတာကို သုံးပြီး အဖြေမှန်အများဆုံး ထုတ်ပေးနိုင်မယ့် parameter တွေရဲ့ တန်ဖိုးကို လေ့လာသင်ယူတာပါပဲ။
- Testing လုပ်တယ် ဆိုတာက ဆောက်ထားတဲ့ မော်ဒယ်ကို မမြင်ဖူးသေးတဲ့ ဒေတာကို ပေးပြီး အဖြေမှန်ကို ခန့်မှန်းခိုင်းတဲ့ အဆင့်ပါပဲ
- Test လုပ်ပြီး ထွက်လာတဲ့ output ကိုမှ evaluation metric တစ်ခုခု ကို သုံးပြီး မော်ဒယ်ကို အမှတ်ပေးတာပါ

Math Notation

We represent the

- Input: $\mathbf{x}$, Output: $\mathbf{y}$
- Normal text ဆိုရင် scalar, bold text ဆိုရင် vector, capital–bold ဆိုရင် matrix
- Structure tabular data ဆိုရင်တော့ $x = [\text{age, mileage}]$
- Function ဆိုရင် အမြဲ လေးထောင့်ကွင်းနဲ့ ဖော်ပြတယ်။ ဥပမာ။ ။ $y = f[x]$
- မော်ဒယ် Parameter တွေ optimization Parameter တွေကိုတော့ ဂရိသင်္ကေတ စာလုံးတွေနဲ့ ကိုယ်စားပြုရေးကြပါတယ်။ ဥပမာ။ ။ $\phi, \alpha, \beta, \theta\lambda$

Key Greek Symbols in Machine Learning

- θ, $\boldsymbol{\theta}$ – model parameters
- ϕ, $\boldsymbol{\phi}$ – feature transformations / parameters
- α – learning rate
- β – coefficients (e.g., linear regression, in $5x$, 5 is the coefficient of x)
- λ – regularization strength
- μ – mean
- σ – standard deviation
- ϵ – error / noise

Why Learn Math Symbols?

- Mathematical equations often look complex and intimidating.
- Example: $\sum_{i=1}^{n} x_i$
- But in reality, they represent simple operations.
- **Key Idea:** Code can help us understand math symbols.
- We can translate equations into Python step by step.

Mathematics as a Language

- Math symbols are like a compact language.
- They summarize operations in a short form.
- Example:
  - Math: $\sum_{i=1}^{n} x_i$
  - Code: loop + accumulation
- **Goal:** Translate symbols into code to understand them.
- ဆရာ့တပည့်တွေ လက်တွေ့လုပ်ကြည့်စေချင်ပါတယ်။

Basic Mathematical Operators

- Addition: $+$ `x + y`
- Subtraction: $-$ `x - y`
- Multiplication: $\times$ `x * y`
- Division: $\div$ `x / y`
- Power: x^2 `x ** 2`

Python Example:

```
x = 2
y = 3
print(x + y)   # 5
print(x ** 2)  # 4
```

Summation Symbol $\sum$

- Math:

$$\sum_{i=1}^{n} x_i$$

- Meaning: Add all elements from $i = 1$ to n

Python Equivalent:

```
x = [1, 2, 3, 4, 5]
result = 0

for i in range(len(x)):
    result += x[i]

print(result)  # 15
```

Product Symbol $\prod$

- Math:

$$\prod_{i=1}^{n} x_i$$

- Meaning: Multiply all elements

Python Equivalent:

```
x = [1, 2, 3, 4]
result = 1

for i in range(len(x)):
    result *= x[i]

print(result)  # 24
```

Factorial $n!$

- Math:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Python Equivalent:

```
result = 1
for i in range(1, 6):
    result *= i

print(result)  # 120
```

Conditional Expressions

- Math uses conditions (piecewise functions)
- Example:

$$f(x) = \begin{cases} 1 & x > 0 \\ 0 & \text{otherwise} \end{cases}$$

Python Equivalent:

```
x = 3

if x > 0:
    result = 1
else:
    result = 0
```

Matrix Operations

- Widely used in Machine Learning
- Math:

$$\mathbf{y} = \mathbf{W}\mathbf{x}$$

Python (NumPy):

```
import numpy as np

W = np.array([[1, 2], [3, 4]])
x = np.array([1, 1])

y = np.dot(W, x)
print(y)
```

Key Takeaways

- Math symbols are compact representations of logic.
- Many symbols correspond directly to code structures:
  - $\sum \rightarrow$ loop + addition
  - $\prod \rightarrow$ loop + multiplication
  - conditions $\rightarrow$ if/else
- **Best way to learn:** Translate math into code.

Exercise

- Convert the following into Python:

$$\sum_{i=1}^{5} i^2$$

- Hint: Use a loop

Solution

- We want to compute:

$$\sum_{i=1}^{5} i^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2$$

Python Implementation:

```python
result = 0

for i in range(1, 6):  # from 1 to 5
    result += i ** 2

print(result)  # Output: 55
```

- Loop iterates from 1 to 5
- Each value is squared and added

The Mathematical Framework

We represent the supervised learning model as a mathematical function:

$$\mathbf{y} = \mathbf{f}[\mathbf{x}] \tag{1}$$

Where:

- $\mathbf{x}$ is the input.
- $\mathbf{y}$ is the output.

However, the model also includes **parameters** ϕ that define the behavior:

$$\mathbf{y} = \mathbf{f}[\mathbf{x}, \phi] \tag{2}$$

The Role of Parameters

- Parameters affect the outcome of the equation.
- The model represents a **family of equations** or **a family of functions**.
- **Training** = Finding the specific parameters ϕ that predict outputs "well" for a training dataset.

1D Linear Regression: The Baseline

The simplest form of supervised learning.

$$\begin{aligned} y &= \mathrm{f}[x, \phi] \\ &= \phi_0 + \phi_1 x \end{aligned} \tag{3}$$

- ϕ_0: The y-intercept (offset/bias). $x = 0$ ဖြစ်တဲ့အချိန်မှာ ရှိတဲ့ Regression line ရဲ့ စမှတ်ပါပဲ။
- ϕ_1: The slope (weight). မျဉ်းက ဘယ်လိုပုံစံနဲ့ စောင်းမလဲ။
- Weight $\rightarrow$ controls importance of input
- Bias $\rightarrow$ shifts output up/down
- မျဉ်းဖြောင့် ဖော်မြူလာ ဖြစ်တဲ့ $y = a + bx$ ကို ပြန်သတိရပါ။

Visualizing Linear Regression

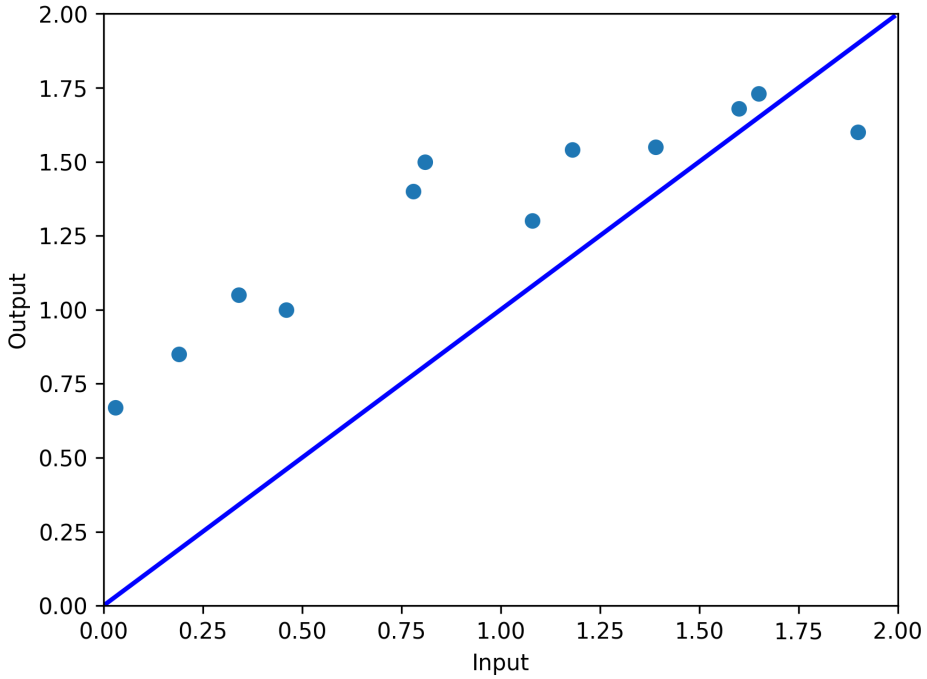

Figure: linear regression with residuals

- Every point (x_i, y_i) in our dataset is compared to the line.
- The vertical distance is the **residual**.

Visualizing Linear Regression

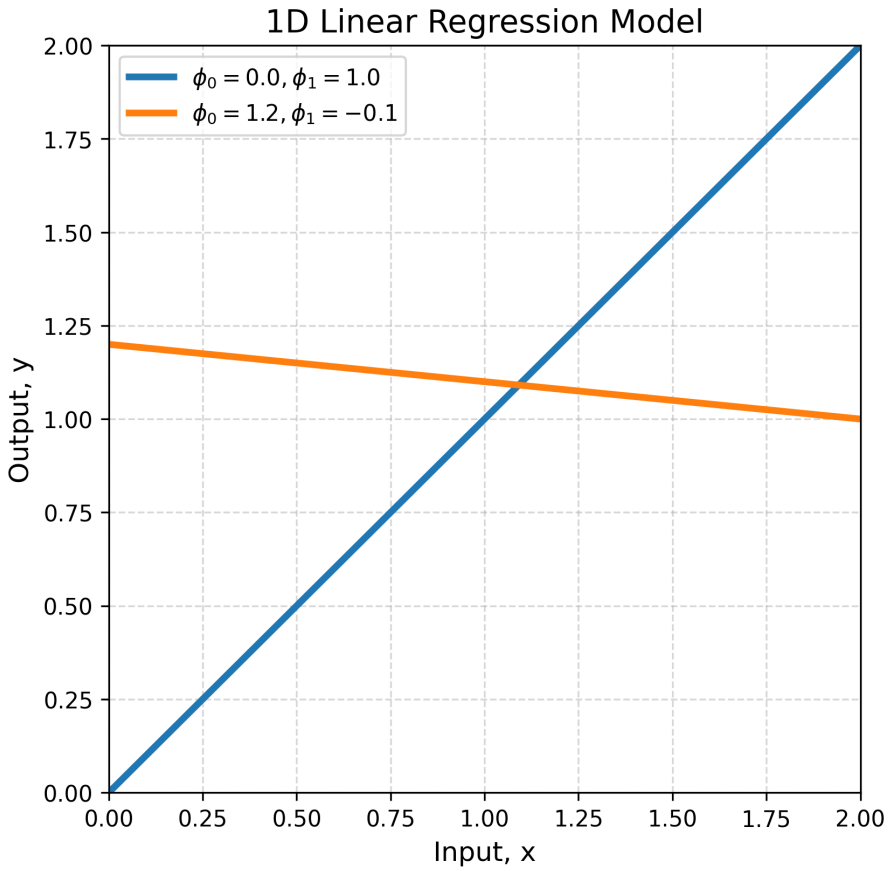

Figure: Two regression lines

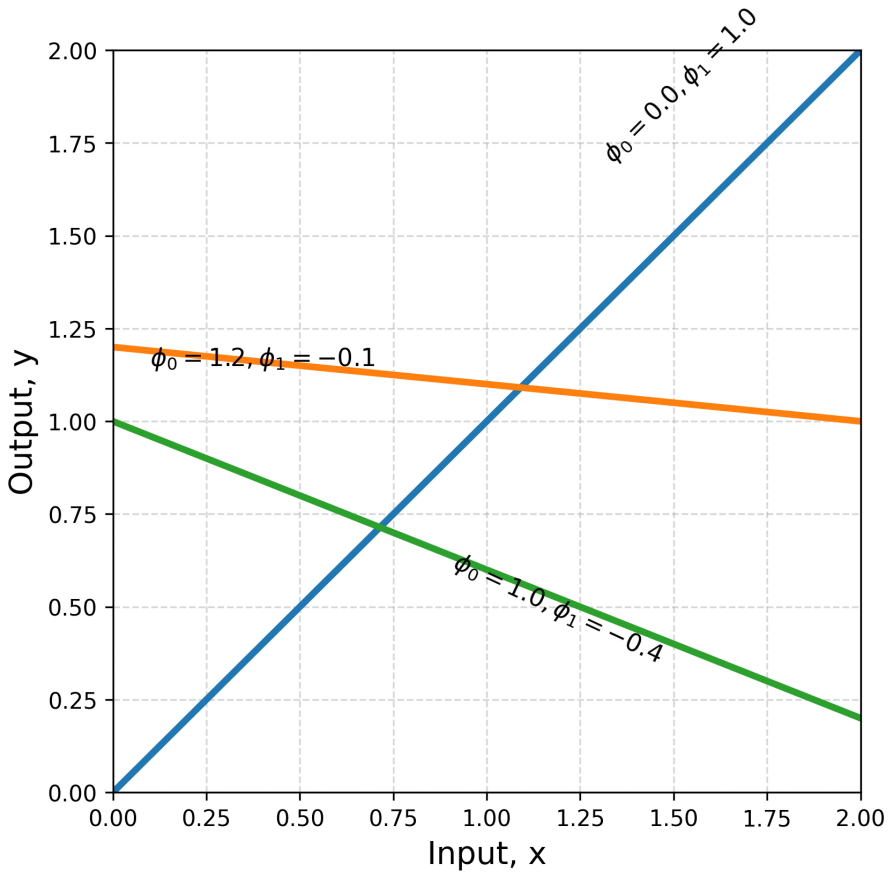

Figure: Three regression lines

The Loss Function

To find the best parameters, we need to quantify the error. We use the Least Squares Loss:

$$
\begin{aligned}
L[\phi] &= \sum_{i=1}^{I} \left(\mathrm{f}[x_i, \phi] - y_i\right)^2 \\
&= \sum_{i=1}^{I} \left(\phi_0 + \phi_1 x_i - y_i\right)^2
\end{aligned} \tag{4}
$$

- Lower loss = Better fit.
- We square the differences to ensure positive values and penalize large outliers.

Training via Optimization

Our goal is to find the set of parameters $\hat{\phi}$ that results in the minimum possible loss:

$$\begin{aligned} \hat{\phi} &= \underset{\phi}{\operatorname{argmin}} \Big[L[\phi] \Big] \\ &= \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^{I} (\phi_0 + \phi_1 x_i - y_i)^2 \right] \end{aligned} \tag{5}$$

The Loss Surface

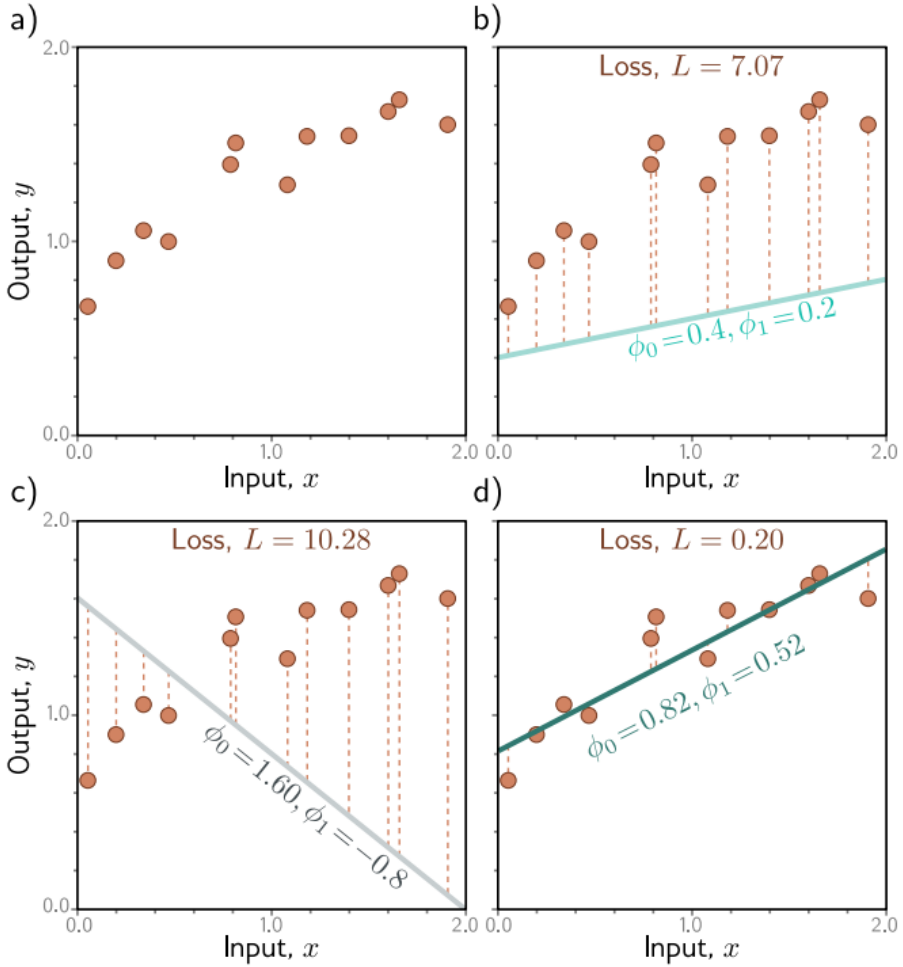

Figure: Linear regression training data, model, and loss. Figure from: https://udlbook.github.io/udlbook/

The Loss Surface

- a) The training data (orange points) consist of $I = 12$ input/output pairs xi, yi. b–d) Each panel shows the linear regression model with different parameters.
- Depending on the choice of y-intercept and slope parameters $\phi = [\phi_0, \phi_1]$, the model errors (orange dashed lines) may be larger or smaller. The loss L is the sum of the squares of these errors.
- The parameters that define the lines in panels (b) and (c) have large losses $L = 7.07$ and $L = 10.28$, respectively because the models fit badly.
- The loss $L = 0.20$ in panel (d) is smaller because the model fits well; in fact, this has the smallest loss of all possible lines, so these are the optimal parameters.

The Loss Surface

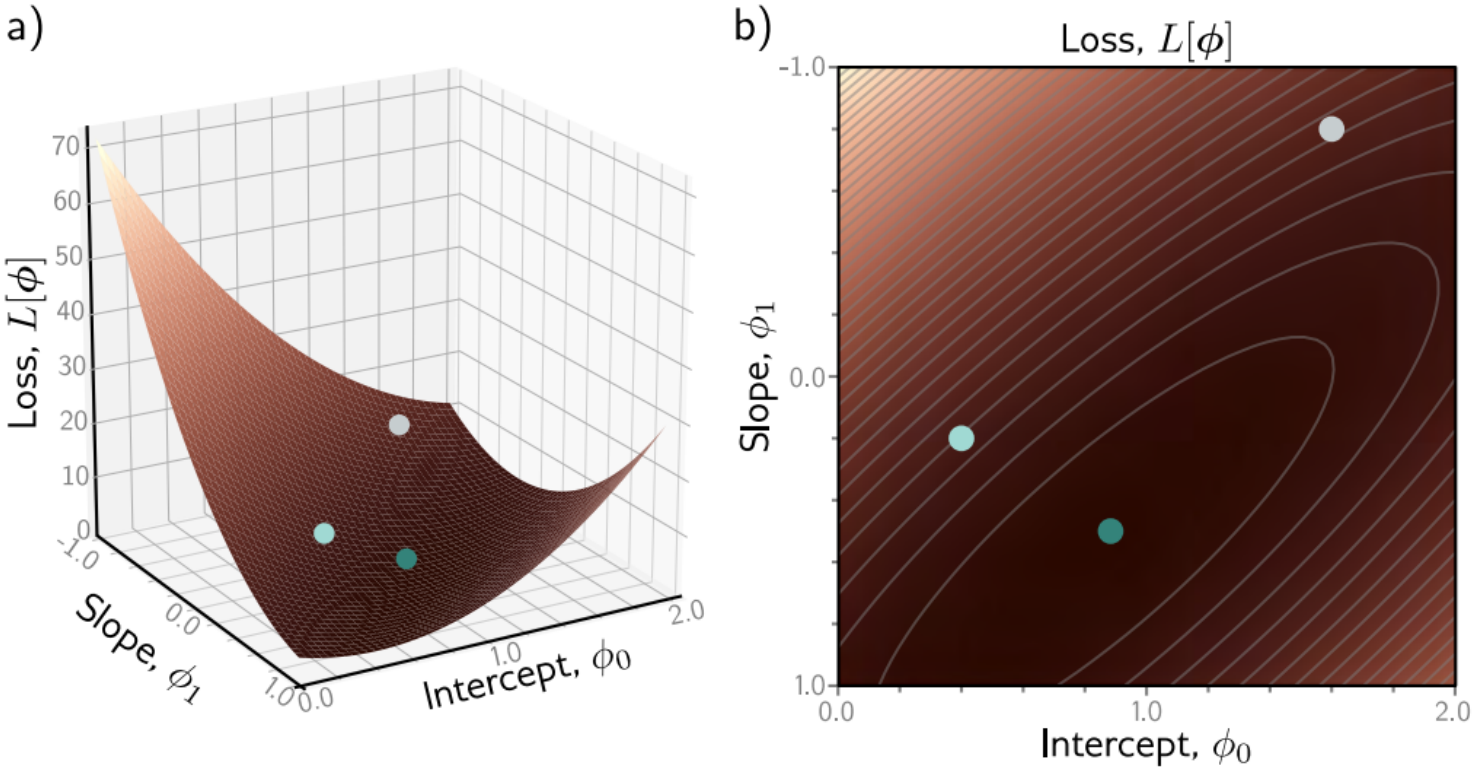

Figure: Loss function for linear regression model with the dataset in previous figure a).
Figure from: https://udlbook.github.io/udlbook/

The Loss Surface

- A bowl-shaped surface
- Axes: $\phi_0 \rightarrow$ intercept, $\phi_1 \rightarrow$ slope
- Height $\rightarrow$ loss $L[\phi]$
- Every point on this surface = one model
- The lowest point of the bowl = best model
- Each pair of parameters creates a line. This surface shows how good each line is.
- Dark region $\rightarrow$ low loss (good)
- Bright region $\rightarrow$ high loss (bad)

Practical Code: Setup

```python
import numpy as np
import matplotlib.pyplot as plt

# Generate synthetic data
x = np.array([0.03, 0.19, 0.34, 0.46, 0.78, 0.81, 1.08, 1.18, 1.39,
    1.60, 1.65, 1.90])
y = np.array([0.06, 0.85, 1.11, 1.11, 1.52, 1.10, 1.67, 1.60, 2.05,
    1.89, 2.10, 1.74])

def f(x, phi0, phi1):
    # The linear model: y = phi0 + phi1 * x
    return phi0 + phi1 * x
```

Practical Code: Calculating Loss

- The ** operator in Python is the exponentiation (power) operator.

```python
def calculate_loss(x, y, phi0, phi1):
    y_pred = f(x, phi0, phi1)
    # Sum of squared differences
    loss = np.sum((y_pred - y) ** 2)
    return loss


# Example calculation
print(f"Loss for phi0=0.5, phi1=0.5: {calculate_loss(x, y, 0.5, 0.5)}")
```

Testing the Model

- A model that fits training data perfectly might fail on new data.
- **Training Set:** Used to adjust ϕ.
- **Test Set:** Unseen data used to evaluate the model's performance in the real world.

Underfitting vs Overfitting

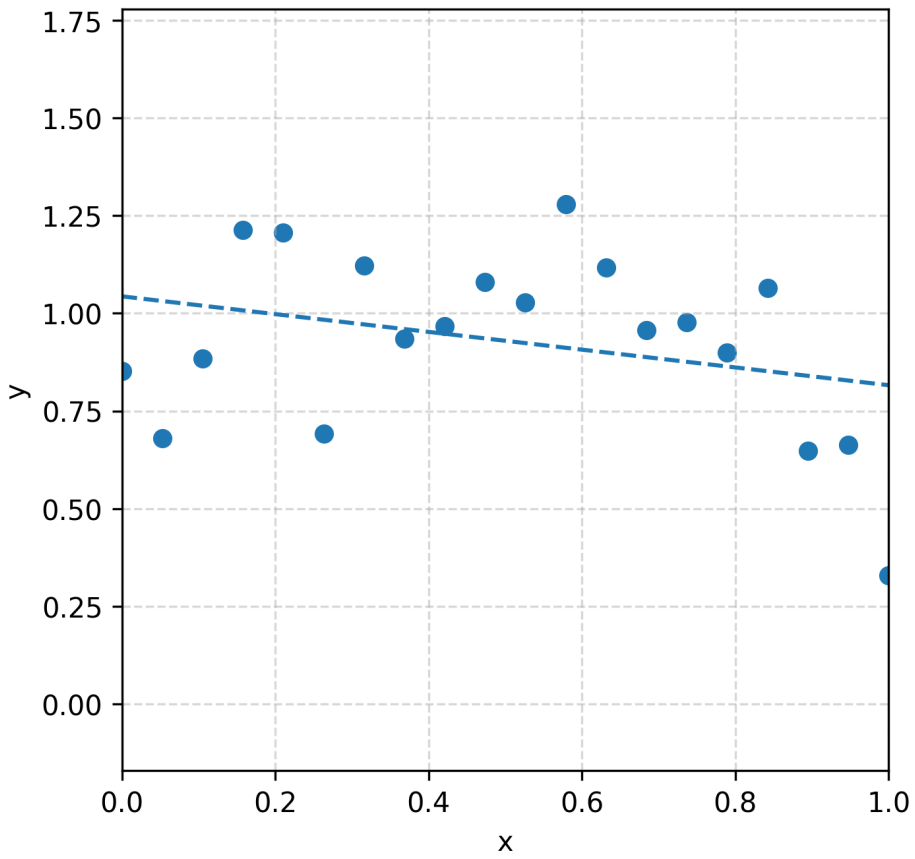

Figure: Underfitting

- **Underfitting:** Model is too simple (e.g., trying to fit a line to a curve).

Underfitting vs Overfitting

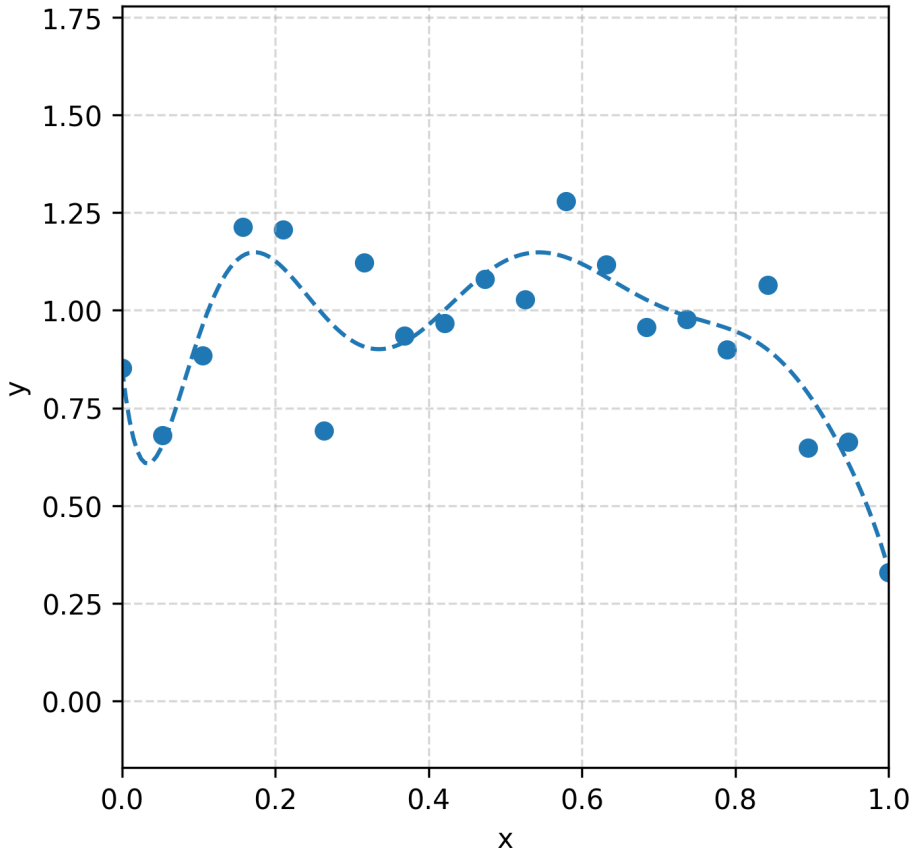

Figure: Overfitting

- **Overfitting:** Model is too complex; it memorizes noise.

Underfitting vs Overfitting

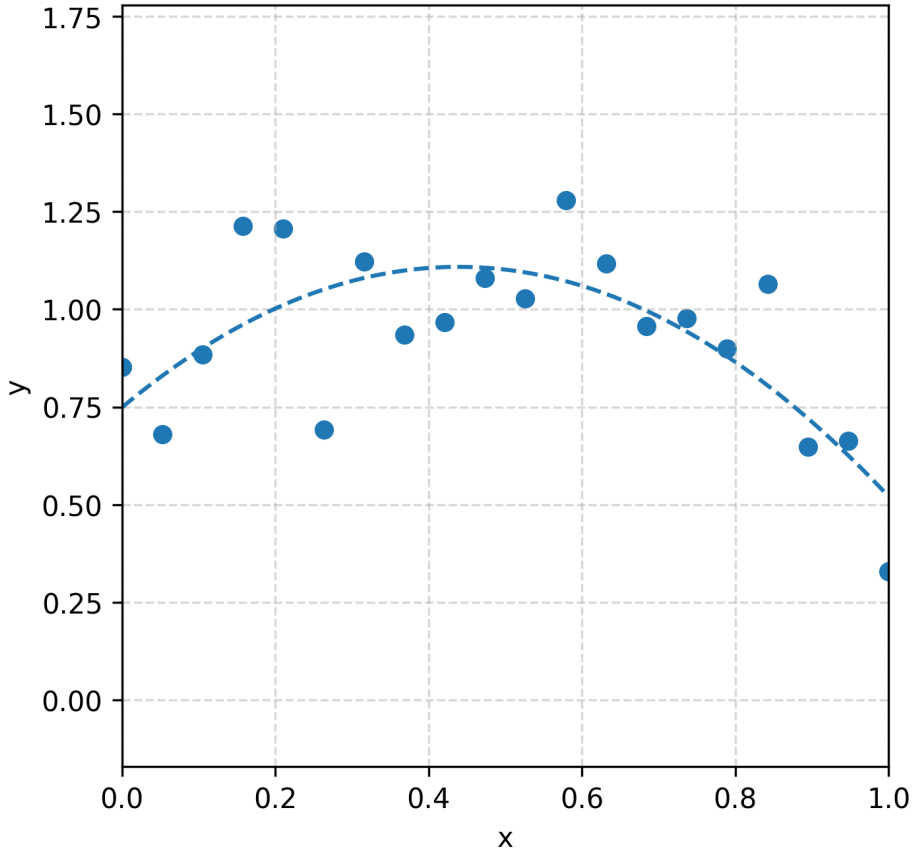

Figure: Good fitting

- **Good Fit:** Model captures the true pattern in the data without fitting the noise.

Limitations of Linear Regression

- Real-world data is rarely a straight line.
- We can extend linear models to multiple inputs (multiple linear regression).
- We can also train them efficiently using optimization methods like SGD.
- We need **non-linear** functions to capture complex patterns.
- This brings us to Shallow and then Deep Neural Networks.

Algorithm: Simple Linear Regression

Goal: Learn parameters ϕ_0, ϕ_1 from data

Pseudo-code

1. Initialize ϕ_0, ϕ_1 (e.g., 0)
2. Repeat until convergence:
   - Compute predictions: $\hat{y}_i = \phi_0 + \phi_1 x_i$
   - Compute loss: $L = \sum_i (\hat{y}_i - y_i)^2$
   - Compute gradients:
     - $\frac{\partial L}{\partial \phi_0} = 2 \sum_i (\hat{y}_i - y_i)$
     - $\frac{\partial L}{\partial \phi_1} = 2 \sum_i (\hat{y}_i - y_i) x_i$
   - Update parameters:
     - $\phi_0 \leftarrow \phi_0 - \alpha \frac{\partial L}{\partial \phi_0}$
     - $\phi_1 \leftarrow \phi_1 - \alpha \frac{\partial L}{\partial \phi_1}$

Algorithm: Multiple Linear Regression

Model: $\hat{y} = \phi_0 + \phi_1 x_1 + \phi_2 x_2 + \cdots + \phi_D x_D$

Pseudo-code

1. Initialize parameters $\boldsymbol{\phi} = [\phi_0, \phi_1, \ldots, \phi_D]$
2. Repeat until convergence:
   - For each data point:
   $$\hat{y}_i = \phi_0 + \sum_{j=1}^{D} \phi_j x_{ij}$$
   - Compute loss:
   $$L = \sum_i (\hat{y}_i - y_i)^2$$
   - Compute gradients for each parameter:
   $$\frac{\partial L}{\partial \phi_j} = 2 \sum_i (\hat{y}_i - y_i) x_{ij}$$
   - Update all parameters:
   $$\phi_j \leftarrow \phi_j - \alpha \frac{\partial L}{\partial \phi_j}$$

Algorithm: Stochastic Gradient Descent (SGD)

Idea: Update parameters using one sample at a time

Pseudo-code

1. Initialize parameters ϕ
2. Repeat for multiple epochs:
   - Shuffle the dataset
   - For each data point (x_i, y_i):
     - Predict:
     $$\hat{y}_i = \phi_0 + \phi_1 x_i$$
     - Compute error:
     $$e_i = \hat{y}_i - y_i$$
     - Compute gradients:
     $$\frac{\partial L}{\partial \phi_0} = 2e_i, \quad \frac{\partial L}{\partial \phi_1} = 2e_i x_i$$
     - Update parameters:
     $$\phi_j \leftarrow \phi_j - \alpha \frac{\partial L}{\partial \phi_j}$$

Comparison: Mathematical Formulation

- **Simple Linear Regression:**

$$\hat{y} = \phi_0 + \phi_1 x$$

  - Single input feature
  - Straight line model
- **Multiple Linear Regression:**

$$\hat{y} = \phi_0 + \sum_{j=1}^{D} \phi_j x_j$$

  - Multiple input features
  - Still a linear model (in parameters)
- **SGD-based Linear Regression:**
  - Same mathematical model as above
  - Different **optimization method** (iterative updates)

Comparison: Algorithmic Perspective

- **Simple Linear Regression (Batch GD):**
  - Uses entire dataset to compute gradients
  - Updates parameters once per iteration
- **Multiple Linear Regression:**
  - Same algorithm structure
  - More parameters $(\phi_1, \ldots, \phi_D)$
  - Higher computational cost
- **Stochastic Gradient Descent (SGD):**
  - Updates parameters using one sample at a time
  - Faster updates, but noisy
  - Often converges faster in practice

Comparison: Pros, Cons, and Applications

- **Simple Linear Regression**
  - **Pros:** Easy to understand, fast
  - **Cons:** Too simple for real-world data
  - **Use:** Basic trend analysis
- **Multiple Linear Regression**
  - **Pros:** Can model multiple factors
  - **Cons:** Still limited to linear relationships
  - **Use:** Economics, healthcare, prediction tasks
- **SGD (for Linear Models)**
  - **Pros:** Scales to large datasets, memory efficient
  - **Cons:** Noisy updates, sensitive to learning rate
  - **Use:** Large-scale ML, online learning

Coding with sklearn

- ဆရာ ဥပမာအနေနဲ့ အောက်ပါ Python code တွေကို ရေးပြထားပါတယ်။
  - `simple_lr.py`
  - `multiple_lr.py`
  - `sgd_lr.py`

Visualization of Multiple Layer Regression

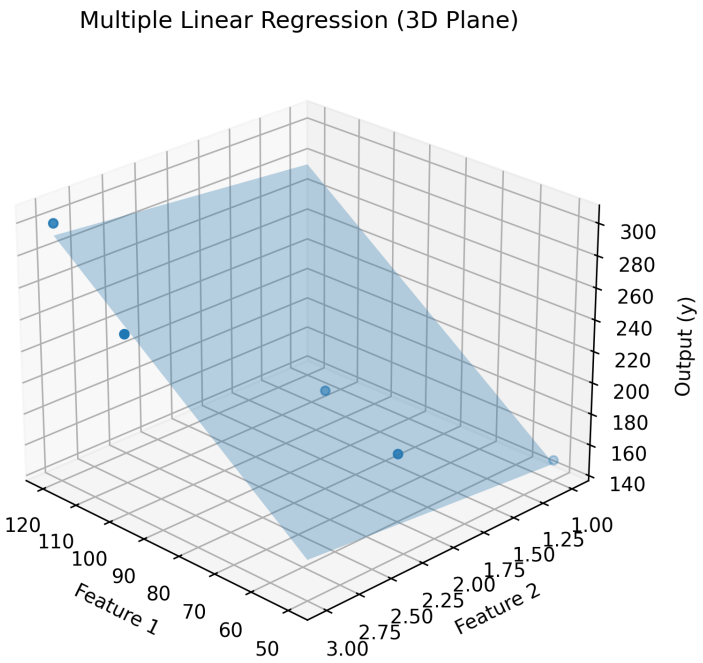

Figure: Multiple linear regression

Visualization of Multiple Layer Regression

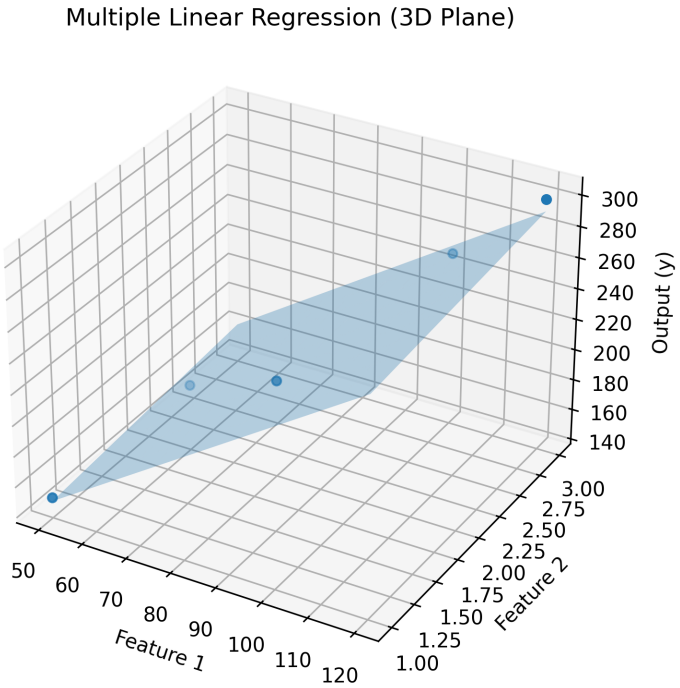

Figure: Multiple linear regression (view.init နဲ့ လှည့်ပြထားတာ)

Evolution of Supervised Learning

1. **Early Models (1960s–1990s):** Linear Regression, Logistic Regression, k-NN, Decision Trees.
2. **Classical ML Era:** Support Vector Machines (SVMs), Random Forests, Gradient Boosting.
3. **Key Idea:** Feature engineering was more important than the model itself.
4. **Deep Learning Era (2010–2020):** Multi-layer Perceptrons, CNNs for vision, RNNs/LSTMs for sequences.
5. **Big Change:** Models started to learn features automatically from raw data.
6. **Modern Era:** Transformers and attention-based models dominate NLP, vision, and speech tasks.
7. **Current Trend:** Larger models + larger datasets = better performance.

R&D Trend: Fairness and Bias

- Supervised learning is only as good as the data labels.
- Research now focuses on De-biasing datasets.
- Human-in-the-loop (RLHF) to align models with human values.

Summary

- Supervised Learning is finding $f[x, \phi]$.
- Training is minimizing the Loss Function.
- Generalization is more important than training accuracy.
- We are moving from simple lines to complex, bio-inspired architectures.

References

Online Resources:

- **Mathematical Notation in ML:**
  https://machinelearningmastery.com/basics-mathematical-notation-machine-learning/
- **Greek Letters in Math & Science:**
  https://en.wikipedia.org/wiki/Greek_letters_used_in_mathematics_science_and_engineering
- **Linear Regression (Overview):**
  https://en.wikipedia.org/wiki/Linear_regression
- **GitHub of UDL Book:**
  https://github.com/udlbook/udlbook
- **Website of Understanding Deep Learning**
  https://udlbook.github.io/udlbook/
- **PDF of Pattern Recognition and Machine Learning (2006)**
  https://github.com/Benlau93/Data-Science-Curriculum/blob/master/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf
- **PDF of Machine Learning A Probabilistic Perspective**
  https://www.sas.upenn.edu/%7Efdiebold/NoHesitations/BookAdvanced.pdf

References

Recommended Books:

- C. M. Bishop, *Pattern Recognition and Machine Learning*
- K. P. Murphy, *Machine Learning: A Probabilistic Perspective*
- T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*
- A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
- Simon J.D. Prince, *Understanding Deep Learning, 2023*
- M. Nielsen, *Neural Networks and Deep Learning*